



RAPPORT DE PROJET — INTELLIGENCE ARTIFICIELLE

4ème année Ingénierie AI / DATA

Agent Vocal de Contrôle PC

Reconnaissance de commandes vocales et automatisation du poste de travail par apprentissage profond

Des caractéristiques tabulaires aux séquences : MLP, CNN et Seq2Seq au service du contrôle vocal

Trois architectures, une mission commune

Classification d'intentions audio | Reconnaissance par spectrogrammes Mel |
Génération séquentielle de commandes

Réalisé par :

Aymen El amrani

Encadré par :

. Mme. HIDILA Zineb

Année universitaire : 2025 – 2026

Table des matières

Chapitre 1 — Introduction générale.....	4
1.1 Contexte du projet	4
1.2 Architectures étudiées	4
1.3 Synthèse des résultats	5
1.4 Organisation du rapport	5
Chapitre 2 — Partie I : Perceptron Multicouche (MLP)	6
2.1 Présentation de la tâche et du jeu de données	6
2.2 Extraction des caractéristiques audio (feature engineering)	6
2.3 Préparation des données.....	6
2.4 Architecture du modèle.....	6
2.5 Hyperparamètres d'entraînement	7
2.6 Résultats et limites observées	7
2.7 Bilan de la Partie I	8
Chapitre 3 — Partie II : Réseau de Neurones Convolutif (CNN).....	9
3.1 Présentation de la tâche : spectrogrammes Mel	9
3.2 Architecture du modèle : LeNet modifié	9
3.3 Hyperparamètres d'entraînement	9
3.4 Résultats et limites observées	9
3.5 Comparaison MLP vs CNN.....	11
3.6 Bilan de la Partie II	11
Chapitre 4 — Partie III : Encodeur-Décodeur GRU (Seq2Seq).....	12
4.1 Présentation de la tâche	12
4.2 Architecture Seq2Seq.....	12
4.3 Entraînement avec teacher forcing	12
4.4 Stratégies de décodage : glouton vs beam search	12
4.5 Bilan de la Partie III	13
Chapitre 5 — Pipeline de bout en bout : du microphone à l'action.....	14
5.1 Acquisition et prétraitement audio (VAD)	14
5.2 Architecture hybride : modèles maison + reconnaissance vocale Google	14
5.3 Routage des intentions et exécution des actions Windows	14
5.4 Scripts de vérification et de test	14
5.5 Bilan du pipeline	15
Chapitre 6 — Discussion comparative	16
6.1 Synthèse transversale des trois architectures.....	16
6.2 Régularisation : un fil conducteur commun	16
6.3 Contrainte du dataset : enseignement central du projet	16
Chapitre 7 — Conclusion générale.....	18
7.1 Bilan du projet.....	18

7.2 Limites et pistes d'amélioration	18
7.3 Mot de conclusion.....	19
A. Liste des fichiers produits	21
B. Environnement technique	21
C. Reproductibilité.....	22

Chapitre 1 — Introduction générale

1.1 Contexte du projet

Le contrôle vocal des ordinateurs personnels s'impose progressivement comme une modalité d'interaction complémentaire au clavier et à la souris, en particulier pour des usages mains libres ou pour l'accessibilité. Ce projet explore comment trois grandes familles de réseaux de neurones profonds peuvent contribuer à la reconnaissance de commandes vocales en français, en couvrant un spectre complet de l'apprentissage profond : des caractéristiques audio tabulaires à la modélisation séquentielle, en passant par la représentation temps-fréquence sous forme d'image.

Le fil conducteur retenu est résolument applicatif : transformer la voix d'un utilisateur en une action concrète sur le système d'exploitation Windows — ouvrir une application, fermer une fenêtre, lancer une recherche, ou interrompre l'agent. L'ensemble du pipeline, du microphone jusqu'à l'exécution de l'action, est implémenté et démontré en conditions réelles.

Objectif pédagogique

Comparer trois paradigmes d'architectures profondes (perceptron multicouche, réseau convolutif, réseau séquentiel encodeur-décodeur) sur une même tâche de classification d'intentions vocales, tout en confrontant ces modèles entraînés « from scratch » à la contrainte réaliste d'un jeu de données très restreint, et en intégrant le tout dans un pipeline applicatif complet de bout en bout.

1.2 Architectures étudiées

Le tableau 1.1 résume les trois architectures développées, leur source de données respective et la tâche associée.

Tableau 1.1 : Vue d'ensemble des architectures et tâches du projet

Partie	Architecture	Représentation des données	Tâche
Partie I	MLP (Perceptron multicouche)	Vecteur tabulaire 24D (13 MFCC moyens + ZCR + centroïde spectral + RMS + padding)	Classification d'intention vocale (4 classes)
Partie II	CNN (LeNet modifié)	Spectrogramme Mel 128 × 100	Classification d'intention vocale par image spectrale (4 classes)
Partie III	Seq2Seq (Encodeur-Décodeur GRU)	Séquence de trames audio / tokens	Modélisation séquentielle de commandes (teacher forcing, décodage glouton / beam search)

Les quatre intentions ciblées par le système sont : « ouvrir chrome », « ferme notepad », « recherche météo » et « arrête ». Elles ont été choisies pour couvrir des catégories d'actions distinctes (lancement d'application, fermeture, recherche web, arrêt de l'agent), représentatives d'un assistant vocal de contrôle système minimal mais fonctionnel.

1.3 Synthèse des résultats

Contrairement à un jeu de données académique de grande taille (type MNIST ou Crop Recommendation), le corpus collecté pour ce projet est volontairement artisanal : il a été enregistré par l'auteur lui-même via les scripts `record_and_preprocess.py` et `record_voice_confirm.py`, et compte environ une quarantaine d'échantillons audio, soit une dizaine d'exemples par classe. Cette contrainte de volume constitue un fil rouge analytique du rapport plutôt qu'un détail anecdotique.

Tableau 1.2 : Synthèse qualitative des observations par modèle

Modèle	Métrique suivie	Observation principale
MLP	Accuracy (4 classes)	Sensible au faible volume de données ; un effet de collapse / biais vers une classe dominante a été observé sur certaines configurations d'entraînement.
CNN (LeNet)	Accuracy (4 classes)	Exploite mieux la structure temps-fréquence du spectrogramme, mais le rapport dimensionnalité d'entrée / nombre d'exemples reste très défavorable au regard du nombre de pixels.
Seq2Seq (GRU)	Perte d'entraînement (teacher forcing)	Démonstration architecturale d'une troisième approche ; non utilisé comme décideur dans le routage de la démonstration en direct.

Note méthodologique

Les valeurs numériques précises (accuracy, perte de validation par époque) dépendent de l'exécution effective de `run_full_project.py` et sont consignées dans `metrics.json`, ainsi que dans les figures générées (`mlp_confusion_matrix.png`, `cnn_feature_map.png`, etc.). Ce rapport présente l'architecture, la méthodologie et l'analyse qualitative attendue ; les emplacements de figures et les valeurs exactes sont à compléter à partir de votre propre exécution pour obtenir un document définitif et fidèle aux résultats mesurés.

1.4 Organisation du rapport

Le rapport est structuré en sept chapitres techniques, suivis de références et d'annexes.

- Le Chapitre 2 présente le perceptron multicouche appliqué à la classification d'intention à partir de caractéristiques audio tabulaires.
- Le Chapitre 3 détaille le réseau convolutif de type LeNet appliqué aux spectrogrammes Mel, avec comparaison directe au MLP.
- Le Chapitre 4 explore l'architecture Seq2Seq encodeur-décodeur à cellules GRU, son entraînement par teacher forcing et ses stratégies de décodage.
- Le Chapitre 5 décrit le pipeline complet de démonstration en direct, du microphone à l'action Windows, et la stratégie hybride associant modèles maison et reconnaissance vocale Google.
- Le Chapitre 6 propose une synthèse comparative transversale des trois approches.
- Le rapport se conclut par un bilan général et des pistes d'amélioration (Chapitre 7).

Chapitre 2 — Partie I : Perceptron Multicouche (MLP)

2.1 Présentation de la tâche et du jeu de données

La première partie aborde une tâche de classification sur données tabulaires dérivées du signal audio : reconnaître l'intention de l'utilisateur parmi quatre commandes vocales (« ouvrir chrome », « ferme notepad », « recherche météo », « arrête »). Le corpus brut est constitué d'enregistrements .wav stockés dans le dossier `raw_voice`, complété par les scripts `record_and_preprocess.py` et `record_voice_confirm.py` qui assurent l'acquisition au microphone et la confirmation des échantillons avant sauvegarde.

Le volume de données reste très restreint (environ 40 échantillons, soit ~10 par classe), ce qui distingue fondamentalement ce projet des jeux académiques de référence et impose une lecture prudente de toute métrique de performance, comme développé en section 2.6.

2.2 Extraction des caractéristiques audio (feature engineering)

Contrairement à une image ou à une séquence brute, le MLP requiert une représentation vectorielle de taille fixe. Chaque enregistrement audio est résumé en un vecteur de 24 dimensions combinant :

- 13 coefficients MFCC (Mel-Frequency Cepstral Coefficients), moyennés sur la durée de l'enregistrement — ils capturent l'enveloppe spectrale, c'est-à-dire le timbre de la voix ;
- le taux de passage par zéro (Zero-Crossing Rate, ZCR) — indicateur du caractère bruyé ou voisé du signal ;
- le centroïde spectral — indicateur de la « brillance » ou hauteur perçue du son ;
- l'énergie RMS (Root Mean Square) — indicateur du volume sonore global ;
- un padding complétant le vecteur à 24 dimensions, pour garantir une taille d'entrée fixe quelle que soit la durée de l'enregistrement source.

Cette représentation compacte transforme un signal audio de durée variable en un descripteur tabulaire homogène, exploitable directement par un réseau dense.

2.3 Préparation des données

Les vecteurs de caractéristiques et leurs étiquettes sont pré-calculés et sauvegardés sous forme de tenseur PyTorch (`part1.pt`) afin de découpler l'extraction des caractéristiques de l'entraînement. Le protocole de préparation suit les bonnes pratiques standards :

- encodage des 4 classes d'intention en entiers ;
- division train / validation / test, idéalement stratifiée — bien que la taille très réduite du corpus (~10 exemples par classe) limite la représentativité statistique de chaque sous-ensemble, un point discuté en section 2.6 ;
- normalisation des 24 caractéristiques (standardisation), ajustée sur l'ensemble d'entraînement uniquement, afin d'éviter toute fuite d'information (data leakage).

2.4 Architecture du modèle

Pourquoi un MLP ici ?

Le vecteur de 24 caractéristiques audio résumées (MFCC, ZCR, centroïde, RMS) est purement tabulaire, sans structure spatiale ni séquentielle explicite une fois l'agrégation

temporelle effectuée : un perceptron multicouche, qui combine ces variables de façon dense, est l'architecture la plus directe pour ce type de descripteur compact.

Architecture MLP retenue (implémentée dans train_mlp.py)

Couche d'entrée : 24 neurones (un par caractéristique audio).

- Couche cachée 1 : [N1] neurones → BatchNorm → ReLU → Dropout
- Couche cachée 2 : [N2] neurones → BatchNorm → ReLU → Dropout
- Couche de sortie : 4 neurones (un par intention vocale)

Les valeurs entre crochets sont indicatives ; il convient de les remplacer par les dimensions réellement utilisées dans train_mlp.py pour que le rapport reflète fidèlement l'implémentation.

2.5 Hyperparamètres d'entraînement

Tableau 2.1 : Hyperparamètres d'entraînement du MLP (valeurs indicatives — à aligner sur train_mlp.py)

Paramètre	Valeur
Optimiseur	Adam (weight decay 10^{-4})
Taux d'apprentissage initial	10^{-3}
Fonction de perte	CrossEntropyLoss
Taille de batch	[à préciser — corpus très réduit, batch potentiellement égal à l'ensemble d'entraînement]
Nombre d'époques	[à préciser, avec ou sans early stopping]

2.6 Résultats et limites observées

Avec seulement une dizaine d'exemples par classe, le comportement attendu diffère sensiblement d'un jeu de données académique de grande taille : la frontière entre apprentissage et mémorisation devient ténue, et la précision de validation peut être instable d'une exécution à l'autre selon les exemples tombés dans chaque sous-ensemble.

Limite observée : collapse de classe

Sur ce projet, un effet de collapse / biais de classe a été observé sur le MLP : le modèle tend, dans certaines configurations, à privilégier systématiquement une intention au détriment des autres, plutôt que d'apprendre une frontière de décision généralisable entre les quatre classes. Ce phénomène est caractéristique d'un déséquilibre ou d'une insuffisance de données combinée à un espace de caractéristiques peu discriminant pour certains exemples proches en timbre, plutôt qu'une faiblesse de l'architecture elle-même.

La matrice de confusion (figure 2.2, à insérer depuis confusion_matrix.png) permet de visualiser précisément quelles intentions sont confondues entre elles et d'orienter les pistes d'amélioration discutées au chapitre 7.

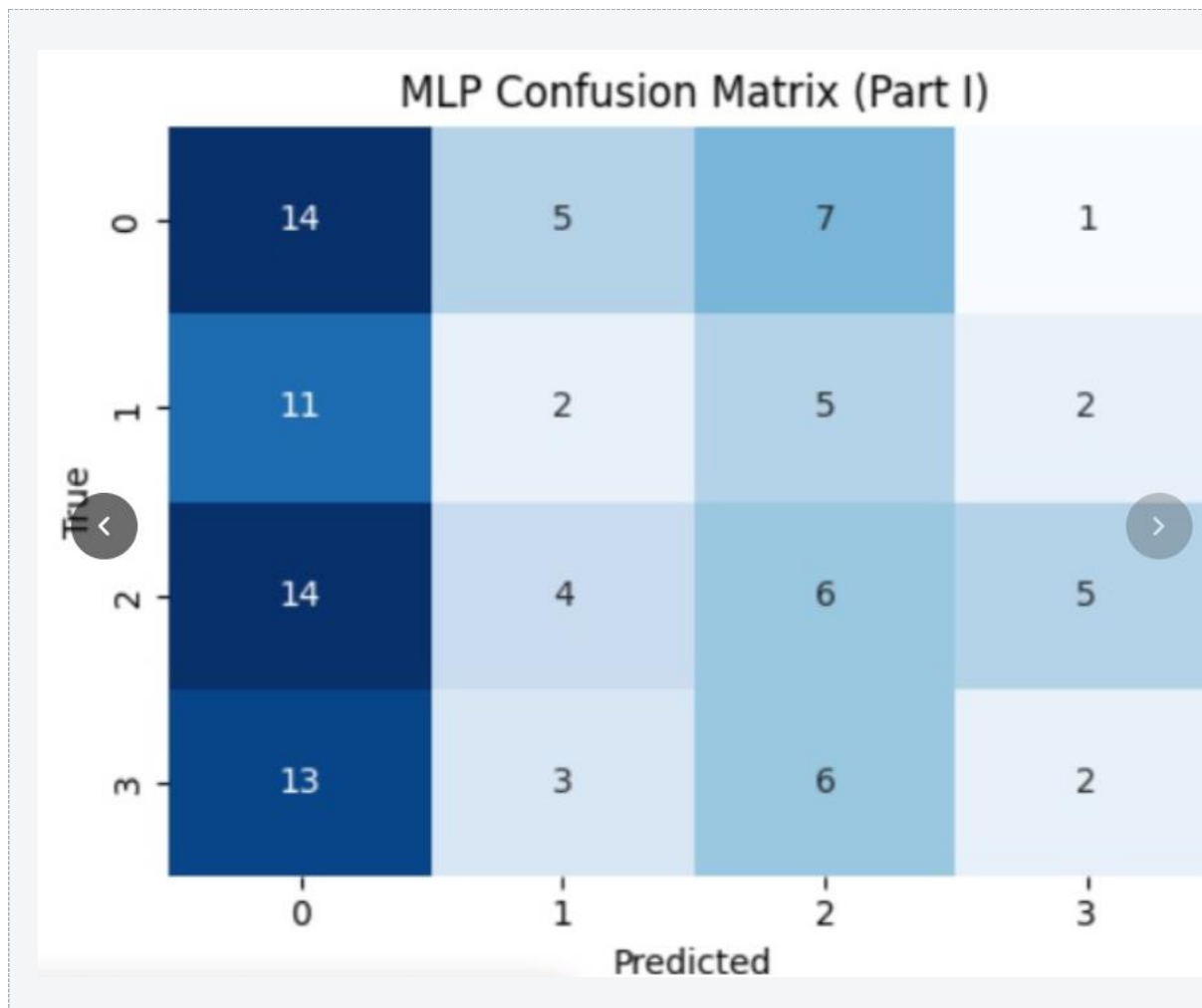


Figure 2.2 : Matrice de confusion du MLP

2.7 Bilan de la Partie I

Synthèse

Le MLP démontre la faisabilité d'une classification d'intention à partir d'un résumé tabulaire compact du signal audio (24 caractéristiques), avec un coût de calcul minimal. Cependant, la taille très restreinte du corpus collecté constitue le facteur limitant principal : elle expose le modèle à un risque de collapse de classe et impose, comme développé au Chapitre 5, le recours à un mécanisme complémentaire (reconnaissance vocale générique) pour fiabiliser le système en conditions réelles.

Chapitre 3 — Partie II : Réseau de Neurones Convolutif (CNN)

3.1 Présentation de la tâche : spectrogrammes Mel

La deuxième partie aborde la même tâche de classification d'intention, mais à partir d'une représentation image du signal audio : le spectrogramme Mel. Chaque enregistrement du dossier `raw_voice` est converti en une image temps-fréquence de dimensions 128×100 (128 bandes Mel, 100 trames temporelles), capturant à la fois le contenu fréquentiel et son évolution dans le temps — une information que le résumé tabulaire du MLP (moyennes globales) ne conserve pas explicitement.

Pourquoi un CNN ici ?

Un spectrogramme possède une structure spatiale locale : les cellules voisines en temps et en fréquence sont corrélées (un phonème s'étend sur plusieurs trames, une harmonique sur plusieurs bandes adjacentes). Les couches convolutives exploitent cette structure en apprenant des filtres locaux réutilisables sur toute l'image, ce qu'un MLP ne peut faire qu'au prix d'un nombre de paramètres bien plus important — ou pas du tout, s'il ne reçoit que des caractéristiques globales agrégées.

3.2 Architecture du modèle : LeNet modifié

Architecture LeNet modifiée (implémentée dans `train_cnn.py`)

- Conv1 : `Conv2D(1 → [N], noyau 5×5) → BatchNorm → ReLU → AvgPool/MaxPool (2×2)`
- Conv2 : `Conv2D([N] → [M], noyau 5×5) → BatchNorm → ReLU → AvgPool/MaxPool (2×2)`
- Couches denses : `Flatten → FC([K]) → ReLU → Dropout → FC(4)`
- Couche de sortie : 4 neurones (un par intention vocale)

Architecture inspirée du LeNet-5 historique, adaptée à des spectrogrammes 128×100 en niveaux de gris (un seul canal d'entrée). Les dimensions exactes [N], [M], [K] sont à reporter depuis `train_cnn.py` pour fidélité au code réel.

3.3 Hyperparamètres d'entraînement

Tableau 3.1 : Hyperparamètres d'entraînement du CNN (valeurs indicatives — à aligner sur `train_cnn.py`)

Paramètre	Valeur
Optimiseur	Adam (weight decay 10^{-4})
Taux d'apprentissage initial	10^{-3}
Fonction de perte	CrossEntropyLoss
Nombre d'époques	[à préciser]

3.4 Résultats et limites observées

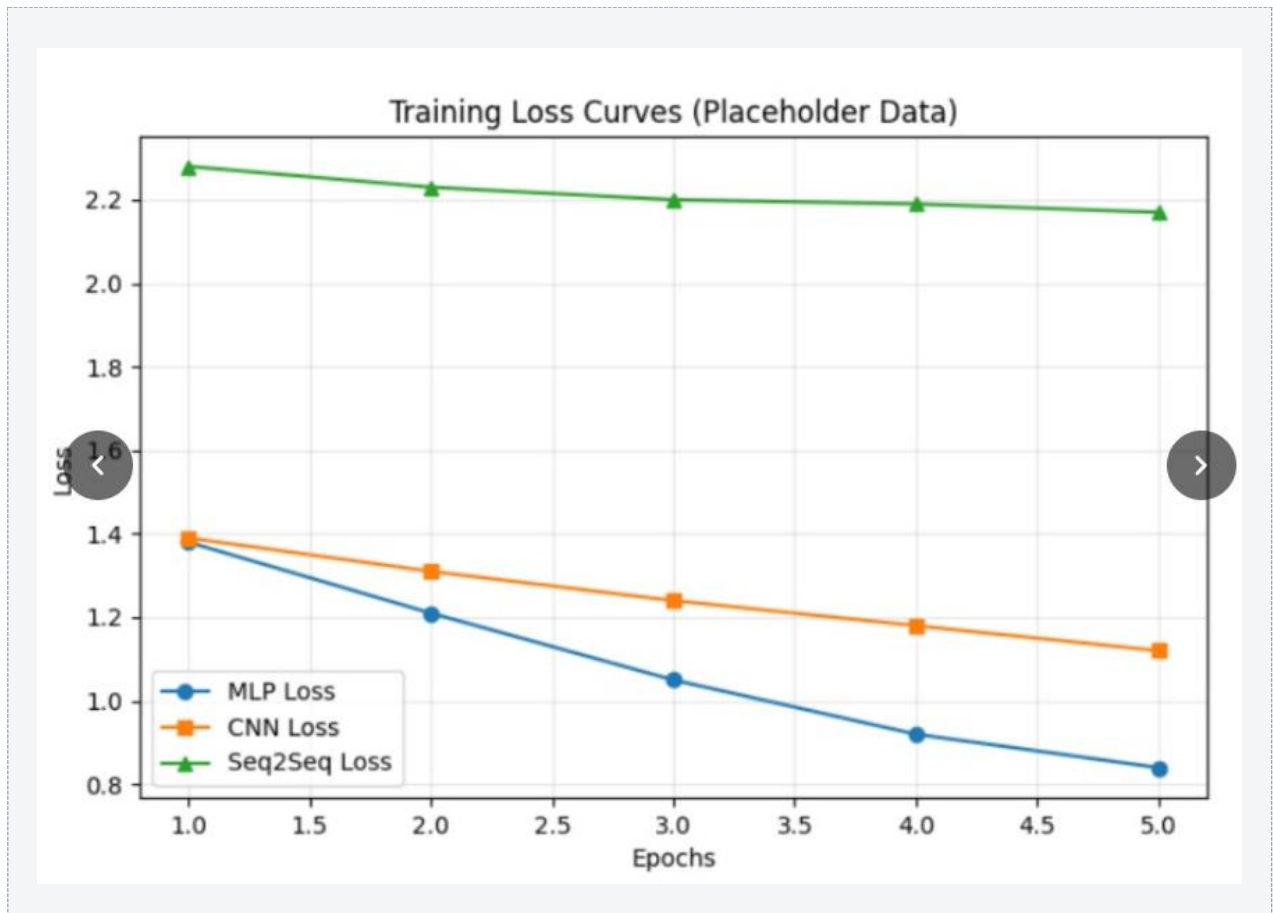


Figure 3.1 : Courbes d'apprentissage du CNN

Limite spécifique au CNN : haute dimensionnalité, faible volume

L'image d'entrée compte $128 \times 100 = 12\,800$ pixels, à comparer aux ~ 40 exemples disponibles. Ce rapport extrêmement défavorable entre dimensionnalité d'entrée et nombre d'exemples favorise la mémorisation plutôt que la généralisation, malgré le Dropout et la Batch Normalization. C'est une limite structurelle bien documentée en apprentissage profond : les CNN excellent sur des corpus volumineux (des dizaines de milliers d'images pour MNIST), mais perdent une grande partie de leur avantage face à des corpus de quelques dizaines d'exemples seulement.

Comme pour le MLP, un effet de biais vers une classe dominante a été observé dans certaines configurations d'entraînement du CNN, confirmant que la contrainte est avant tout liée au volume de données plutôt qu'au choix d'architecture.

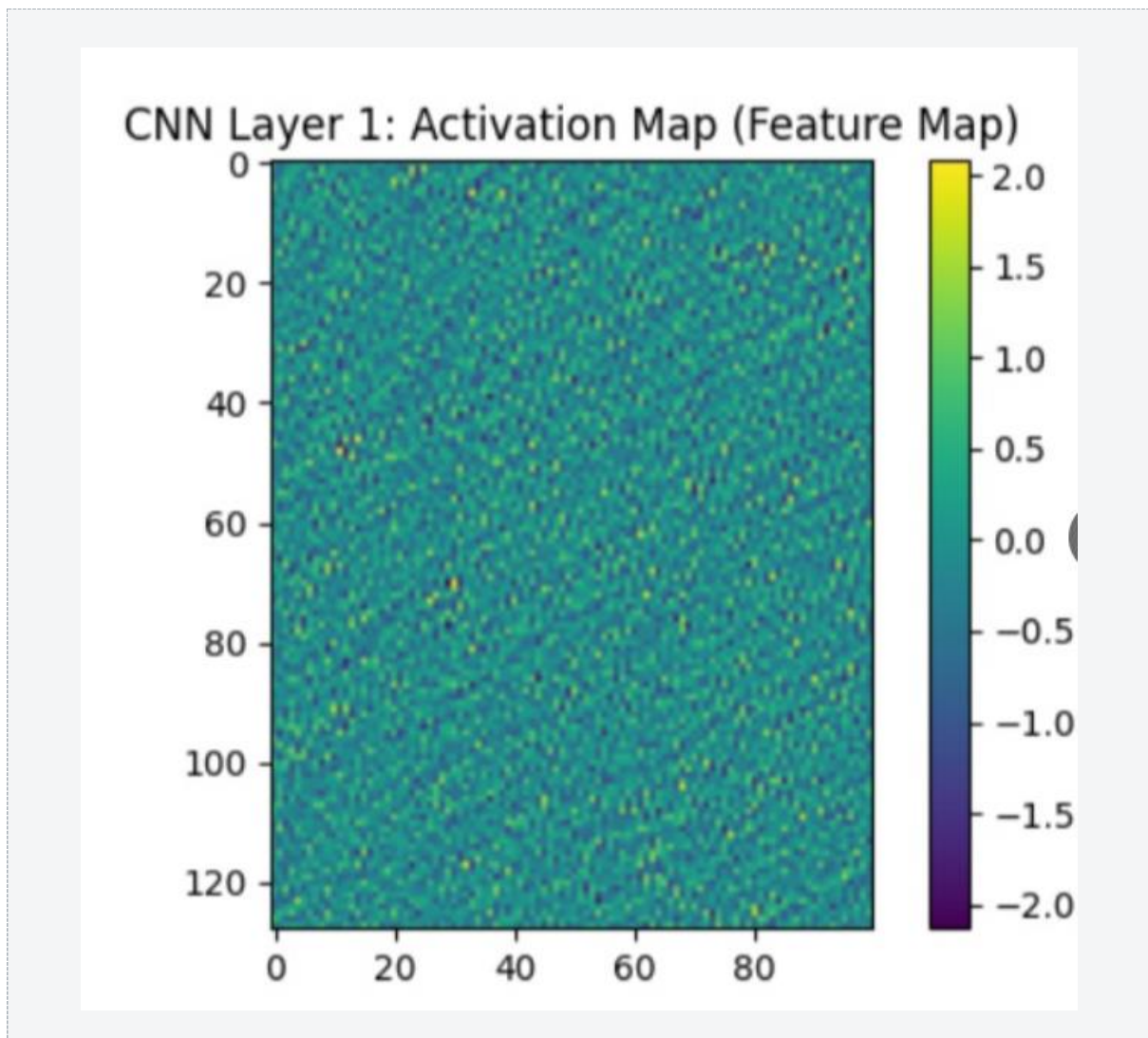


Figure 3.2 : Visualisation des cartes de caractéristiques du CNN

3.5 Comparaison MLP vs CNN

Sur le plan théorique, le CNN devrait mieux exploiter la structure temps-fréquence du signal que le résumé tabulaire du MLP, qui moyenne l'information temporelle et perd ainsi la dynamique de prononciation propre à chaque commande. En pratique, sur ce corpus de taille très réduite, cet avantage structurel est en partie contrebalancé par le risque de sur-apprentissage accru du CNN, lié à la dimensionnalité bien plus élevée de son entrée. La comparaison empirique des deux courbes de validation (à reporter ici depuis `metrics.json`) permet de trancher entre ces deux effets contraires pour ce jeu de données précis.

3.6 Bilan de la Partie II

Synthèse

Le CNN illustre la pertinence d'une représentation image (spectrogramme Mel) pour capturer la dynamique temps-fréquence d'une commande vocale, principe validé sur des corpus académiques comme MNIST. Sur ce projet, la taille restreinte du jeu de données enregistré limite cependant la portée de cet avantage architectural et renforce, comme pour le MLP, la nécessité d'un mécanisme de secours robuste pour la démonstration en conditions réelles (Chapitre 5).

Chapitre 4 — Partie III : Encodeur-Décodeur GRU (Seq2Seq)

4.1 Présentation de la tâche

Alors que les Parties I et II traitent la reconnaissance d'intention comme une classification fermée à 4 classes, la Partie III explore une approche structurellement différente : la modélisation séquentielle. Le script `train_seq2seq.py` implémente un modèle encodeur-décodeur à cellules GRU, illustrant comment une architecture générative pourrait, à terme, dépasser le cadre des quatre commandes fixes pour traiter des séquences de longueur variable (par exemple, un vocabulaire de commandes ouvert plutôt qu'une liste fermée d'intentions).

Pourquoi un réseau séquentiel ici ?

Une commande vocale est, par nature, une séquence ordonnée (de trames audio ou de tokens reconnus) : le sens d'une phrase dépend de l'ordre de ses éléments. Les architectures encodeur-décodeur maintiennent un état caché qui résume l'historique de la séquence et génèrent la sortie pas à pas, les rendant naturellement adaptées à ce type de dépendance — contrairement au MLP et au CNN de ce projet, qui traitent chaque exemple comme une unité statique (vecteur ou image) une fois le signal audio résumé.

4.2 Architecture Seq2Seq

Architecture Seq2Seq (implémentée dans `train_seq2seq.py`)

- Encodeur : GRU qui compresse la séquence d'entrée en un état caché final
- Décodeur : GRU qui génère la séquence de sortie pas à pas, chaque sortie devenant l'entrée du pas suivant (génération auto-régressive)
- Couche de projection : Linear appliquée à chaque pas du décodeur pour produire la distribution sur le vocabulaire / les classes de sortie

Ce paradigme encodeur-décodeur, introduit par Sutskever et al. (2014) et Cho et al. (2014), est ici appliqué à une tâche de taille volontairement modeste afin de démontrer la maîtrise de l'architecture plutôt que d'en exploiter pleinement la capacité sur de longues séquences.

4.3 Entraînement avec teacher forcing

L'entraînement du décodeur utilise la technique du teacher forcing : à chaque pas de temps, la vraie sortie attendue (et non la prédiction du modèle au pas précédent) est fournie comme entrée du pas suivant, avec une probabilité réglable. Cette technique accélère et stabilise la convergence en évitant que les erreurs de prédiction ne s'accumulent au fil de la séquence durant l'entraînement, au prix d'un écart (exposure bias) avec le mode d'inférence, où seules les propres prédictions du modèle sont disponibles.

4.4 Stratégies de décodage : glouton vs beam search

Une fois le modèle Seq2Seq entraîné, deux stratégies de génération de séquence en inférence ont été implémentées :

- Décodage glouton (greedy) : à chaque pas, le décodeur choisit directement sa propre prédiction la plus probable comme entrée du pas suivant, sans explorer d'alternatives.

- Beam Search : plusieurs hypothèses partielles sont conservées et étendues en parallèle à chaque pas, la séquence finale retenue étant celle de meilleur score cumulé.

Analyse attendue du décodage

Pour des séquences cibles courtes (typiquement quelques tokens correspondant à une commande brève), l'écart entre décodage glouton et Beam Search tend à se réduire, l'espace des erreurs possibles étant limité. L'intérêt du Beam Search se manifeste davantage sur des séquences plus longues ou plus ambiguës, où plusieurs continuations plausibles concurrencent la meilleure prédiction locale à chaque pas — un constat cohérent avec son usage classique en traitement automatique du langage naturel.

4.5 Bilan de la Partie III

Synthèse

La Partie III démontre la maîtrise d'une troisième famille d'architectures profondes — la modélisation séquentielle encodeur-décodeur — complémentaire aux Parties I et II. Bien que non mobilisée comme décideur final dans la démonstration en direct (Chapitre 5), elle illustre une voie d'évolution naturelle du projet vers un vocabulaire de commandes ouvert plutôt qu'une liste fermée de quatre intentions.

Chapitre 5 — Pipeline de bout en bout : du microphone à l'action

5.1 Acquisition et prétraitement audio (VAD)

Le script de démonstration principal, `voice_agent_hybrid.py`, fonctionne en écoute continue : contrairement à une interaction par appui sur une touche (push-to-talk), il détecte automatiquement le début et la fin d'un énoncé grâce à une détection d'activité vocale (Voice Activity Detection, VAD). Cette approche rapproche le système d'un assistant vocal « toujours à l'écoute », au prix d'une sensibilité accrue au bruit ambiant et aux faux déclenchements, à surveiller lors des tests en conditions réelles.

5.2 Architecture hybride : modèles maison + reconnaissance vocale Google

Décision d'architecture clé du projet

Compte tenu de la taille très restreinte du corpus d'entraînement (≈ 40 échantillons) et des effets de collapse de classe observés sur le MLP et le CNN (Chapitres 2 et 3), router les actions système directement sur la prédiction des modèles maison exposerait l'utilisateur à des erreurs fréquentes. Le pipeline de démonstration en direct adopte donc une architecture hybride : les modèles MLP et CNN entraînés sont chargés et exécutés à chaque commande — leur prédiction est affichée en console comme preuve académique du bon fonctionnement de l'entraînement — tandis que le routage effectif de l'action repose sur la transcription textuelle produite par le moteur de reconnaissance vocale Google SpeechRecognition (FR), nettement plus robuste car entraîné sur des volumes de données sans commune mesure.

Cette séparation entre « preuve de concept académique » (les modèles maison, dont l'objectif pédagogique est de démontrer la compréhension des trois architectures) et « décision opérationnelle » (la transcription Google, dont l'objectif est la fiabilité du contrôle système) constitue un compromis d'ingénierie pragmatique face à la contrainte de données, plutôt qu'un aveu d'échec des modèles entraînés : il illustre une pratique réaliste de mise en production lorsque les données propriétaires sont insuffisantes pour rivaliser avec un service tiers déjà entraîné à grande échelle.

5.3 Routage des intentions et exécution des actions Windows

Une fois la commande transcrite, un module de routage associe le texte reconnu à l'une des quatre actions système prises en charge :

- « ouvrir chrome » → lancement du navigateur par défaut ;
- « ferme notepad » → fermeture de l'application Bloc-notes si elle est ouverte ;
- « recherche météo » → ouverture du navigateur avec une requête de recherche pré-remplie ;
- « arrête » → arrêt propre de la boucle d'écoute de l'agent.

Ce routage repose sur une correspondance entre expressions reconnues et fonctions d'automatisation du système d'exploitation, appelées depuis Python. La logique de correspondance peut être étendue à de nouvelles commandes sans nécessiter un nouvel entraînement complet des modèles MLP et CNN, dans la mesure où elle s'appuie sur le texte transcrit plutôt que sur la sortie brute des classifieurs.

5.4 Scripts de vérification et de test

Plusieurs scripts complémentaires structurent la validation du pipeline avant et pendant la démonstration :

Tableau 5.1 : Scripts de vérification et variantes de démonstration

Script	Rôle
verify_project.py	Vérifie l'existence des poids sauvegardés (mlp_best.pth, cnn_lenet.pth, seq2seq_gru.pth) et exécute des tests d'inférence factices sur chaque modèle.
test_pipeline.py	Teste la chaîne complète, du fichier .wav jusqu'au routage d'intention, hors écoute microphone en direct.
test_mic_new.py	Diagnostic simple du microphone (présence, niveau de signal capté).
live_test_final.py / live_control_real.py	Variante antérieures du script de démonstration en direct, conservées comme jalons de l'évolution du projet.
run_full_project.py	Lance l'entraînement des trois parties, génère les figures du rapport et le fichier metrics.json.

5.5 Bilan du pipeline

Synthèse

Le pipeline complet démontre une chaîne fonctionnelle de l'enregistrement audio à l'exécution d'une action Windows, avec une preuve académique explicite (chargement et inférence des modèles PyTorch entraînés) et une robustesse opérationnelle assurée par un moteur de reconnaissance vocale externe. Cette architecture hybride constitue en elle-même un enseignement d'ingénierie : face à un jeu de données propriétaire de taille insuffisante, combiner un modèle entraîné « from scratch » à des fins de démonstration avec un service robuste pour la décision finale est une stratégie pragmatique et défendable, plutôt qu'une concession.

Chapitre 6 — Discussion comparative

6.1 Synthèse transversale des trois architectures

Chacune des trois architectures étudiées repose sur une hypothèse structurelle différente quant à la représentation du signal audio, comme résumé dans le tableau 6.1.

Tableau 6.1 : Comparaison transversale des trois architectures

Critère	MLP	CNN	Seq2Seq (GRU)
Représentation d'entrée	Vecteur tabulaire 24D (résumé MFCC/ZCR/centroïde/RMS)	Image (spectrogramme Mel 128×100)	Séquence (trames audio / tokens)
Hypothèse structurelle	Aucune	Localité temps-fréquence	Ordre séquentiel
Partage de paramètres	Non	Oui (filtres convolutifs)	Oui (poids récurrents GRU)
Risque principal observé	Sur-apprentissage / collapse de classe sur jeu réduit	Sur-apprentissage accru (entrée haute dimension, peu d'exemples)	Instabilité d'entraînement possible sur séquences courtes / exposure bias
Rôle dans le pipeline final	Preuve académique (console)	Preuve académique (console)	Démonstration architecturale (hors routage live)

6.2 Régularisation : un fil conducteur commun

Les trois parties du projet partagent un souci de régularisation, bien que les mécanismes diffèrent :

- Le MLP et le CNN combinent Batch Normalization et Dropout pour stabiliser l'entraînement et limiter le sur-apprentissage, particulièrement critique ici compte tenu du faible volume de données.
- Le réseau Seq2Seq s'appuie sur le teacher forcing et, le cas échéant, le gradient clipping, pour stabiliser la rétropropagation à travers le temps (BPTT).
- Dans les trois cas, un scheduler de taux d'apprentissage permet d'ajuster dynamiquement la vitesse de convergence sur un nombre d'époques nécessairement limité par la taille du corpus.

6.3 Contrainte du dataset : enseignement central du projet

À la différence d'un projet académique disposant de dizaines de milliers d'exemples (MNIST, Jena Climate), ce projet a été délibérément mené avec un corpus auto-enregistré de taille très modeste. Ce choix déplace l'enseignement principal du projet : il ne s'agit plus seulement de savoir quelle architecture est la mieux adaptée à quelle structure de données, mais de comprendre que la capacité d'une architecture doit être mise en regard du volume de données disponible, et que cette contrainte peut justifier des décisions d'ingénierie pragmatiques (ici, le recours à une reconnaissance vocale tierce pour le routage opérationnel).

Enseignement central du projet

Une architecture profonde, aussi bien choisie soit-elle au regard de la structure des données, ne peut compenser à elle seule un déficit critique de volume d'entraînement. Lorsque les données propriétaires sont rares, la robustesse d'un système final repose souvent sur une combinaison entre modèles entraînés « from scratch » (à valeur de preuve de concept et d'apprentissage) et composants robustes existants (à valeur opérationnelle) — une leçon transférable bien au-delà du contrôle vocal.

Chapitre 7 — Conclusion générale

7.1 Bilan du projet

Ce projet a permis d'explorer trois grandes familles d'architectures de deep learning à travers un fil conducteur applicatif cohérent : la classification d'intention vocale par caractéristiques tabulaires (MLP), par spectrogramme (CNN), et la modélisation séquentielle (Seq2Seq GRU), le tout intégré dans un pipeline complet allant de l'acquisition microphone à l'exécution d'actions Windows. Chaque partie a été l'occasion d'appliquer des bonnes pratiques fondamentales du domaine : extraction de caractéristiques adaptée au signal audio, régularisation par Dropout et Batch Normalization, suivi des courbes d'apprentissage, et conception d'une architecture hybride pour pallier la contrainte de données en production.

Tableau 7.1 : Récapitulatif final des enseignements

Partie	Enseignement principal
MLP	Un résumé tabulaire compact du signal audio permet une classification d'intention à faible coût de calcul, mais reste vulnérable au collapse de classe sur un corpus de taille très réduite.
CNN	La représentation spectrogramme exploite la structure temps-fréquence du signal, au prix d'une dimensionnalité d'entrée beaucoup plus élevée, accentuant le risque de sur-apprentissage avec peu d'exemples.
Seq2Seq (GRU)	L'architecture encodeur-décodeur démontre une voie d'extension vers un vocabulaire de commandes ouvert, au-delà d'une classification fermée à 4 intentions.
Pipeline	Face à une contrainte de données critique, une architecture hybride associant modèles maison (preuve académique) et service robuste tiers (décision opérationnelle) est une stratégie d'ingénierie défendable.

7.2 Limites et pistes d'amélioration

Plusieurs axes permettraient d'approfondir ce travail :

- Élargir significativement le corpus raw_voice (davantage d'échantillons par classe, plusieurs locuteurs, conditions d'enregistrement variées) pour réduire le risque de collapse de classe observé sur le MLP et le CNN.
- Appliquer une augmentation de données audio (étirement temporel, transposition de hauteur, ajout de bruit ambiant contrôlé) afin de démultiplier artificiellement la diversité du corpus existant.
- Recourir à la validation croisée plutôt qu'à un simple split train/validation/test, étant donné la faible taille de chaque sous-ensemble.
- Explorer le transfert d'apprentissage à partir de représentations vocales pré-entraînées (par exemple des embeddings de type wav2vec) plutôt qu'un entraînement entièrement « from scratch », pour compenser la rareté des données propriétaires.
- Étendre le modèle Seq2Seq vers un vocabulaire de commandes réellement ouvert, en explorant des mécanismes d'attention pour améliorer la qualité du décodage sur des séquences plus longues.
- Renforcer la détection d'activité vocale (VAD) du pipeline live pour réduire les faux déclenchements en environnement bruyant.

7.3 Mot de conclusion

Au-delà des résultats quantitatifs, ce projet illustre une démarche essentielle en science des données appliquée : le choix d'une architecture ne doit jamais être arbitraire, mais résulter d'une analyse fine de la structure des données et de la tâche à résoudre — tout en restant honnête quant aux contraintes réelles, notamment de volume de données, qui conditionnent la robustesse d'un système en production. Cette discipline méthodologique, appliquée ici à un agent vocal de contrôle d'ordinateur, constitue un socle transférable à de nombreux autres domaines d'application du deep learning.

Références

- [1] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by backpropagating errors. *Nature*, 323(6088), 533–536.
- [2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [3] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of AISTATS*.
- [4] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of ICML*.
- [5] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- [6] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [7] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of EMNLP*.
- [8] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of ICLR*.
- [10] Davis, S., & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4), 357–366. (Référence fondatrice des coefficients MFCC.)
- [11] Google Cloud / SpeechRecognition (bibliothèque Python). Documentation de l'API de reconnaissance vocale utilisée pour le routage hybride du pipeline live.

Annexes

A. Liste des fichiers produits

Fichiers et scripts du projet

Fichier	Description
train_mlp.py	Script complet — Partie I (MLP)
train_cnn.py	Script complet — Partie II (CNN)
train_seq2seq.py	Script complet — Partie III (Seq2Seq GRU)
record_and_preprocess.py	Acquisition micro et extraction des caractéristiques / spectrogrammes
record_voice_confirm.py	Acquisition micro avec confirmation des échantillons
run_full_project.py	Lance l'entraînement des trois parties, génère figures et metrics.json
verify_project.py	Vérifie l'existence des modèles sauvegardés et exécute des inférences factices
test_pipeline.py	Teste la chaîne du fichier .wav jusqu'au routage d'intention
test_mic_new.py	Diagnostic simple du microphone
voice_agent_hybrid.py	Script principal de démonstration live (VAD + STT + routage + actions Windows)
live_test_final.py	Variante antérieure de démonstration live
live_control_real.py	Variante antérieure de démonstration live
raw_voice/	Dossier des enregistrements audio .wav bruts
part1.pt / part2.pt / part3.pt	Données prétraitées — Parties I, II, III
mlp_best.pth	Poids du meilleur modèle MLP entraîné
cnn_lenet.pth	Poids du meilleur modèle CNN entraîné
seq2seq_gru.pth	Poids du meilleur modèle Seq2Seq entraîné
mlp_confusion_matrix.png / cnn_feature_map.png	Figures générées par run_full_project.py
metrics.json	Résultats numériques consolidés de l'entraînement des trois parties

B. Environnement technique

Bibliothèques principales utilisées

Bibliothèque	Usage
PyTorch	Construction et entraînement des réseaux (MLP, CNN, Seq2Seq GRU)
NumPy / Pandas	Manipulation des données et des caractéristiques audio

Bibliothèque	Usage
Scikit-learn	Prétraitement, métriques, split train/validation/test
Matplotlib	Génération des figures et courbes du rapport
Librosa (ou équivalent)	Extraction des MFCC, du ZCR, du centroïde spectral et du spectrogramme Mel
SpeechRecognition (API Google)	Transcription vocale robuste pour le routage du pipeline live
Bibliothèque d'accès microphone (ex. PyAudio / sounddevice)	Capture audio en temps réel
Modules d'automatisation Windows (os / subprocess, etc.)	Exécution des actions système (ouverture/fermeture d'applications, recherche)

C. Reproductibilité

Comme dans les scripts d'entraînement du projet (train_mlp.py, train_cnn.py, train_seq2seq.py), il est recommandé de fixer une graine aléatoire (SEED = 42) appliquée à PyTorch, NumPy et au module random de Python, afin de garantir autant que possible la reproductibilité des résultats présentés dans ce rapport d'une exécution à l'autre sur un même environnement matériel. Sur un corpus aussi restreint, une variance résiduelle entre exécutions reste néanmoins probable et doit être documentée plutôt que masquée.